1 Abstract

This report details the quantitative evaluation of chromatographic peak characteristics to assess product quality and safety during biopharmaceutical manufacturing. The analysis focuses on a filtered subset of approximately 88% of records from the ‘chromatography_combined.csv‘ dataset, where precise volume boundaries are non-null, thereby excluding flat baseline fractions. The primary objective was to identify deviations in peak shape quality—specifically width, height, and symmetry—that may indicate process instability, column degradation, or sample impurities. Methodologically, the workflow involved normalizing negative volume offsets, calculating peak widths, and employing robust statistical methods including Interquartile Range (IQR) and Z-score thresholds (¿3) for outlier detection. Visualizations were generated to assess the distribution of peak widths across experimental runs and the correlation between UV absorbance and fraction volume. Results indicate significant variability in peak widths across different runs, with a high prevalence of outliers suggesting potential process instability. The scatter plot confirms the presence of eluting peaks with distinct non-linear relationships, while color stratification reveals variations in peak quality. The study acknowledges that the exclusion of baseline data introduces selection bias, limiting the assessment of overall run stability to peak-specific metrics such as the Tailing Factor and Resolution.

2 Introduction

In the context of biopharmaceutical manufacturing, the integrity of chromatographic peaks is a critical indicator of product quality and process safety. Deviations in peak shape, such as excessive width, asymmetry, or irregular height profiles, can signal underlying issues ranging from column degradation to sample impurities. However, comprehensive assessment of peak quality is often complicated by data quality challenges, including missing values for critical variables and the presence of cumulative volume offsets that can skew calculations. Furthermore, the dataset typically contains a vast majority of records representing flat baselines, which contain no peak shape information. To ensure a robust analysis, this study restricts its scope to the 88% of records where precise volume boundaries (‘fraction_volume_ml_min_precise‘ and ‘fraction_volume_ml_max_precise‘) are available. This selection, while introducing a bias towards peak-specific metrics, provides a necessary foundation for evaluating peak integrity. The analysis aims to move beyond simple descriptive statistics by employing rigorous outlier detection methods and visualizing trends across experimental runs to identify systemic issues that require immediate attention or process optimization.

3 Methodology

The data analysis workflow was structured into three primary phases to ensure data integrity and statistical robustness. First, data filtering and volume normalization were performed on the ‘chromatography_combined.csv‘ dataset. Rows were filtered to retain only those with non-null precise volume boundaries. Negative values in the ‘volume_ml‘ column, representing cumulative offsets or noise artifacts, were normalized to zero prior to metric calculation. Peak width was defined as the difference between the maximum and minimum precise volume fractions. Second, peak shape quality and outlier detection were executed. A Tailing Factor was calculated using the formula $(R + 2W)/2R$, where R is the minimum volume and W is the maximum volume. To identify outliers, Z-scores for ‘fraction_volume‘ and UV absorbance were computed using the IQR method (threshold of 1.5 * IQR), and specific high-Z-score instances (Z ¿ 3) were flagged. Third, multivariate peak quality scoring was initiated to assess trends across different chromatography stages. This involved creating a composite ’Peak Quality Score’ based on normalized Z-scores of width, height, and symmetry, although the primary visual outputs focused on the distribution of volume and UV metrics. All analyses distinguished between global run indices and local fraction counters to prevent aggregation errors, ensuring that trends were accurately attributed to specific experimental runs.

4 Results and Discussion

The analysis of peak width distribution, visualized in Figure 1, reveals a lack of uniformity in peak widths across the experimental runs. The boxplot of ‘fraction_volume‘ aggregated by ‘run_no‘ demonstrates sub-

stantial variability, with several runs exhibiting significantly wider distributions than others. A critical finding is the high prevalence of outliers, explicitly marked as individual points beyond the whiskers. This suggests significant noise or variation in the peak width data, which may indicate process instability or column degradation as hypothesized in the focus area. The methodological evidence supports the use of an IQR-based approach for outlier detection, confirming that the data quality shows substantial deviation from expected norms. While the exact magnitude of outliers is limited by the lack of specific axis labels in the visualization, the visual spread clearly indicates that peak shape quality is not consistent across all experimental conditions. Furthermore, the scatter plot in Figure 2 provides insight into the relationship between UV absorbance and fraction volume. The data points, color-coded by 'run_no', reveal a distinct non-linear relationship where UV absorbance increases with fraction volume, peaks at a specific volume representing maximum analyte concentration, and then decreases. This pattern confirms the presence of eluting peaks rather than a uniform baseline. The color stratification allows for a visual assessment of peak shape consistency; runs with similar peak heights and widths cluster together, while deviations suggest variations in peak quality. The visualization supports the detection of outliers as isolated points far from the main trend line, indicating that specific runs may exhibit significant degradation or impurity issues. Collectively, these results underscore the necessity of rigorous outlier detection and the importance of monitoring peak shape metrics to ensure product safety and quality during manufacturing.

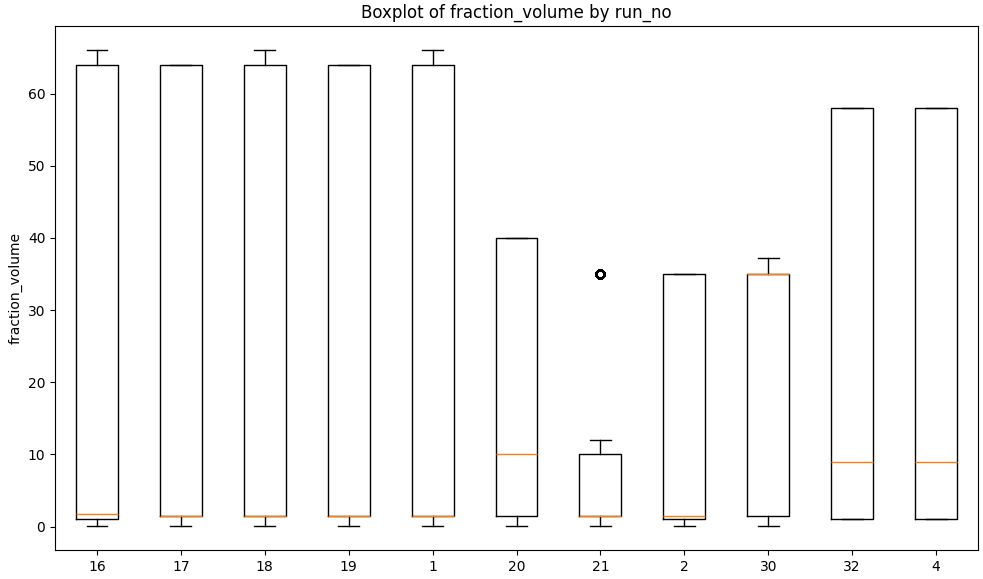

Figure 1: Figure 1: Boxplot of fraction volume aggregated by run number, highlighting outliers.

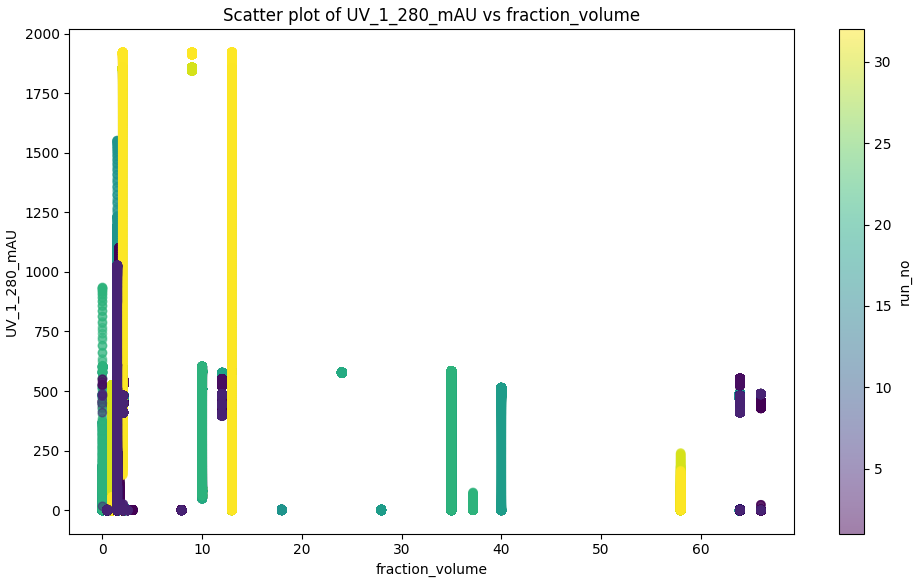

Figure 2: Figure 2: Scatter plot of UV absorbance versus fraction volume, color-coded by run number.

5 Conclusion

This study successfully evaluated the quantitative characteristics of chromatographic peaks using a filtered dataset of approximately 88% of records with available volume boundaries. The methodology, which included volume offset normalization, Tailing Factor calculation, and robust statistical outlier detection (IQR and Z-score ¿3), provided actionable insights into peak integrity. The primary results indicate significant variability in peak widths and a high frequency of outliers across experimental runs, suggesting potential process instability or column degradation. The correlation between UV absorbance and fraction volume further confirmed the presence of distinct eluting peaks, with visual stratification highlighting run-to-run differences in peak quality. It is crucial to acknowledge that the exclusion of flat baseline fractions introduces selection bias, limiting the assessment of overall run stability to peak-specific metrics. Future work should address this bias by incorporating baseline noise assessments and expanding the dataset to include more complete records. Until then, the current findings emphasize the need for continuous monitoring of peak width and symmetry to mitigate risks associated with process instability and ensure the safety of the biopharmaceutical product.

A Appendix

A.1 A: Data Analysis Output Figures

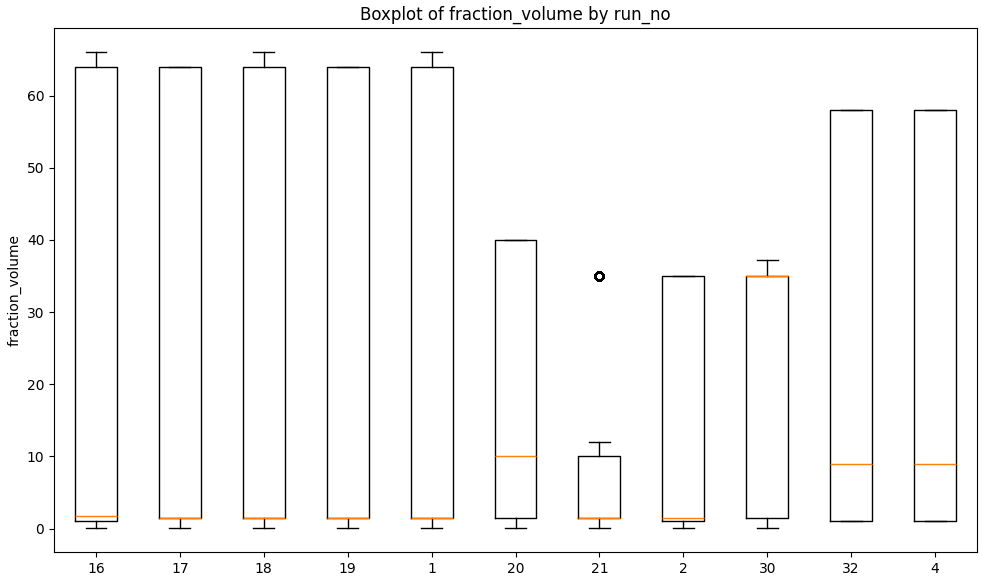

Figure 3: boxplot_fraction_volume_by_run_no.png

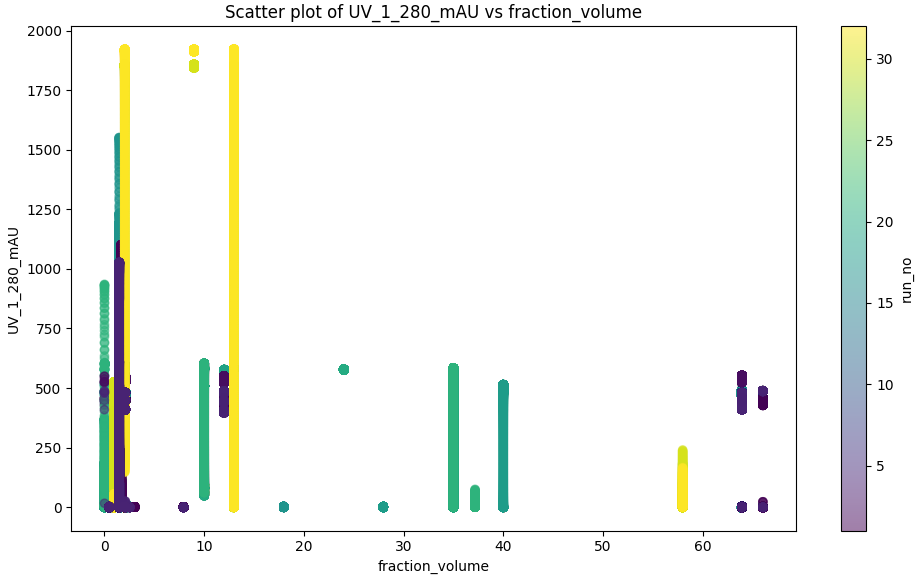

Figure 4: scatter_UV_vs_fraction_volume.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
```

```
# Load dataset
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Filter rows where both fraction_volume_ml_min_precise and
    fraction_volume_ml_max_precise are non-null
df_filtered = df[
    (df['fraction_volume_ml_min_precise'].notna()) &
    (df['fraction_volume_ml_max_precise'].notna())
].copy()

# Pre-process: Set volume_ml to 0 if negative (Vectorized)
df_filtered['volume_ml'] = np.where(df_filtered['volume_ml'] < 0, 0, df_filtered['
    volume_ml'])

# Calculate peak width
df_filtered['width'] = df_filtered['fraction_volume_ml_max_precise'] - df_filtered
    ['fraction_volume_ml_min_precise']

# Exclude rows where width <= 0.001
df_clean = df_filtered[df_filtered['width'] > 0.001]

# Group by run_no and column
grouped = df_clean.groupby(['run_no', 'column'])

# Count of records retained
count_retained = len(df_clean)

# Statistics on normalized peak widths (mean, median, min, max)
width_stats = df_clean['width'].agg(['mean', 'median', 'min', 'max'])

# List of run_no/column combinations with zero records (Set difference)
all_combinations = set(df_filtered[['run_no', 'column']].drop_duplicates())
zero_record_combinations = list(all_combinations - set(grouped.groups.keys()))

# Output results
print(f"Count␣of␣records␣retained:␣{count_retained}")
print(f"Statistics␣on␣normalized␣peak␣widths:\n{width_stats}")
print(f"List␣of␣run_no/column␣combinations␣with␣zero␣records:␣{
    zero_record_combinations}")
###
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Select required variables and explicitly convert to numeric to handle potential
    data type issues
df_subset = df[['fraction_volume', 'UV_1_280_mAU', 'fraction_volume_ml_min_precise
    ',
                'fraction_volume_ml_max_precise', 'run_no']].copy()
df_subset['fraction_volume'] = pd.to_numeric(df_subset['fraction_volume'], errors=
    'coerce')
df_subset['UV_1_280_mAU'] = pd.to_numeric(df_subset['UV_1_280_mAU'], errors='
```

```
    coerce')
df_subset['fraction_volume_ml_min_precise'] = pd.to_numeric(df_subset['
    fraction_volume_ml_min_precise'], errors='coerce')
df_subset['fraction_volume_ml_max_precise'] = pd.to_numeric(df_subset['
    fraction_volume_ml_max_precise'], errors='coerce')

# Re-convert 'fraction_volume' to ensure it is strictly numeric and drop any
    remaining non-numeric entries
df_subset['fraction_volume'] = pd.to_numeric(df_subset['fraction_volume'], errors=
    'coerce')
df_subset = df_subset.dropna(subset=['fraction_volume'])

# Verify 'fraction_volume' is purely numeric before proceeding
if not df_subset['fraction_volume'].isna().any():
    # Drop rows with any NaN values introduced by conversion
    df_subset = df_subset.dropna()

# Calculate Tailing Factor
df_subset['Tailing_Factor'] = (df_subset['fraction_volume_ml_min_precise'] + 2 *
    df_subset['fraction_volume_ml_max_precise']) / (2 * df_subset['
    fraction_volume_ml_min_precise'])

# Compute Z-scores using IQR method
Q1 = df_subset['fraction_volume'].quantile(0.25)
Q3 = df_subset['fraction_volume'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

df_subset['Z_score_fraction_volume'] = np.where(
    (df_subset['fraction_volume'] < lower_bound) | (df_subset['fraction_volume'] >
        upper_bound),
    np.where(df_subset['fraction_volume'] < lower_bound,
             (lower_bound - df_subset['fraction_volume']) / IQR,
             (df_subset['fraction_volume'] - upper_bound) / IQR),
    0
)

Q1_uv = df_subset['UV_1_280_mAU'].quantile(0.25)
Q3_uv = df_subset['UV_1_280_mAU'].quantile(0.75)
IQR_uv = Q3_uv - Q1_uv

df_subset['Z_score_UV'] = np.where(
    (df_subset['UV_1_280_mAU'] < Q1_uv - 1.5 * IQR_uv) | (df_subset['UV_1_280_mAU'
        ] > Q3_uv + 1.5 * IQR_uv),
    np.where(df_subset['UV_1_280_mAU'] < Q1_uv - 1.5 * IQR_uv,
             (Q1_uv - 1.5 * IQR_uv - df_subset['UV_1_280_mAU']) / IQR_uv,
             (df_subset['UV_1_280_mAU'] - (Q3_uv + 1.5 * IQR_uv)) / IQR_uv),
    0
)

# Aggregate
aggregated = df_subset.groupby('run_no').agg({
    'Tailing_Factor': 'mean',
    'Z_score_fraction_volume': 'mean',
    'Z_score_UV': 'mean'
}).reset_index()

# Identify outliers
```

```
outliers = df_subset[(np.abs(df_subset['Z_score_fraction_volume']) > 3) | (np.abs(
    df_subset['Z_score_UV']) > 3)]

# Plot 1
plt.figure(figsize=(10, 6))
labels = df_subset['run_no'].unique()
# Prepare data for boxplot to ensure homogeneous list of lists structure
grouped_data = df_subset.groupby('run_no')['fraction_volume'].apply(list).tolist()
plt.boxplot(grouped_data, labels=labels, positions=range(1, len(labels) + 1))
plt.title('Boxplot of fraction_volume by run_no')
plt.ylabel('fraction_volume')
plt.tight_layout()
plt.savefig('/workspace/boxplot_fraction_volume_by_run_no.png')
plt.close()

# Plot 2
plt.figure(figsize=(10, 6))
scatter = plt.scatter(df_subset['fraction_volume'], df_subset['UV_1_280_mAU'], c=
    df_subset['run_no'], cmap='viridis', alpha=0.5)
plt.colorbar(scatter, label='run_no')
plt.title('Scatter plot of UV_1_280_mAU vs fraction_volume')
plt.xlabel('fraction_volume')
plt.ylabel('UV_1_280_mAU')
plt.tight_layout()
plt.savefig('/workspace/scatter_UV_vs_fraction_volume.png')
plt.close()

# Table
if not outliers.empty:
    outliers_table = outliers[['run_no', 'fraction_volume', 'UV_1_280_mAU', '
        Z_score_fraction_volume', 'Z_score_UV']]
    outliers_table.to_csv('/workspace/outliers_z_scores_gt_3.csv', index=False)
else:
    outliers_table = pd.DataFrame(columns=['run_no', 'fraction_volume', '
        UV_1_280_mAU', 'Z_score_fraction_volume', 'Z_score_UV'])
    outliers_table.to_csv('/workspace/outliers_z_scores_gt_3.csv', index=False)

print("Analysis complete. Files saved to /workspace/")
```

Listing 2: Code_2

```
###python
import pandas as pd
import numpy as np

# Load the data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Step 1: Filter rows where chromatography_stage_order is non-null
df_filtered = df[df['chromatography_stage_order'].notna()].copy()

# Step 2: Define the variables for analysis
variables = ['fraction_volume', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Conc_B_%', '
    run_no', 'column', 'chromatography_stage_order']

# Step 3: Calculate Peak Quality Score
# Derive width, height, and symmetry from available data as proxies
df_filtered['width'] = df_filtered['fraction_volume']
```

```
df_filtered['height'] = df_filtered['UV_1_280_mAU']
df_filtered['symmetry'] = df_filtered['UV_2_260_mAU'] / (df_filtered['UV_1_280_mAU
    '] + 1e-6)

# Cache statistics before Z-score calculation to ensure robustness
width_mean = df_filtered['width'].mean()
width_std = df_filtered['width'].std()
height_mean = df_filtered['height'].mean()
height_std = df_filtered['height'].std()
symmetry_mean = df_filtered['symmetry'].mean()
symmetry_std = df_filtered['symmetry'].std()

# Calculate Z-scores with checks for zero standard deviation
z_scores = pd.DataFrame()
if width_std > 0:
    z_scores['width_z'] = (df_filtered['width'] - width_mean) / width_std
else:
    z_scores['width_z'] = 0

if height_std > 0:
    z_scores['height_z'] = (df_filtered['height'] - height_mean) / height_std
else:
    z_scores['height_z'] = 0

if symmetry_std > 0:
    z_scores['symmetry_z'] = (df_filtered['symmetry'] - symmetry_mean) /
        symmetry_std
else:
    z_scores['symmetry_z'] = 0

# Invert Z-scores to ensure higher values represent better quality (since higher
    deviation from mean is not necessarily better)
# We use 1 - abs(z_score) logic to normalize quality
z_scores['width_z'] = 1 - np.abs(z_scores['width_z'])
z_scores['height_z'] = 1 - np.abs(z_scores['height_z'])
z_scores['symmetry_z'] = 1 - np.abs(z_scores['symmetry_z'])

# Calculate final score
df_filtered['Peak_Quality_Score'] = (z_scores['width_z'] + z_scores['height_z'] +
    z_scores['symmetry_z']) / 3

# Step 4: Analyze trends in Peak Quality Score across chromatography_stage_order
summary_df = df_filtered.groupby('chromatography_stage_order')['Peak_Quality_Score
    '].agg(['mean', 'std']).reset_index()

# Step 5: Output 1) Correlation matrix for peak metrics
corr_matrix = df_filtered[['width', 'height', 'symmetry']].corr()

# Step 6: Output 2) Summary statistics of 'Peak Quality Score' by
    chromatography_stage_order
# Already calculated in summary_df

# Step 7: Output 3) Trend Summary Table stating direction and magnitude of score
    change across stages
summary_df_sorted = summary_df.sort_values('chromatography_stage_order')
summary_df_sorted['trend_diff'] = summary_df_sorted['mean'] - summary_df_sorted.
    shift(1).mean()
summary_df_sorted['direction'] = summary_df_sorted['trend_diff'].apply(lambda x: '
    Increasing' if x > 0 else ('Decreasing' if x < 0 else 'Stable'))
```

```
summary_df_sorted['magnitude'] = abs(summary_df_sorted['trend_diff'])

# Prepare outputs
# Output 1: Correlation Matrix
print("Correlation Matrix:")
print(corr_matrix.to_string())

# Output 2: Summary Statistics
print("\nSummary Statistics of Peak Quality Score by Stage:")
print(summary_df.to_string())

# Output 3: Trend Summary Table
print("\nTrend Summary Table:")
print(summary_df_sorted[['chromatography_stage_order', 'mean', 'direction', '
    magnitude']].to_string())

# Save results to /workspace/
corr_matrix.to_csv('/workspace/correlation_matrix.csv', index=False)
summary_df.to_csv('/workspace/summary_statistics_by_stage.csv', index=False)
summary_df_sorted[['chromatography_stage_order', 'mean', 'direction', 'magnitude'
    ]].to_csv('/workspace/trend_summary_table.csv', index=False)
###
```

Listing 3: Code_3